# SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

**TIRUCHIRAPPALLI - 621105**

**B.Tech in Information Technology**

2026-2027

Course Code: 21CSC301T

**Course Name: FORMAL LANGUAGE AND AUTOMATA**

Report on

# FORMALIZING SMART-HOME ENERGY AUTOMATION SYSTEMS: SIMULATION AND ANALYSIS USING FINITE AUTOMATA

Submitted by

Name of the Student : S Sanjay
Roll Number : RA2411008050002

Under the guidance of

**Dr. Nagaraj P**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

Name of the Student : G Venkatesh
Roll Number : RA2411008050057

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)
TIRUCHIRAPPALLI

## TABLE OF CONTENTS


ABSTRACT . . . . . 3

1. INTRODUCTION & THEORETICAL MOTIVATION . . . . . 3

1.1 Problem Statement . . . . . 3

1.2 The Case for Formal Language and Automata Theory (FLA) . . . . . 3

2. LITERATURE SURVEY & THEORETICAL FOUNDATION . . . . . 4

2.1 Classical Automata in Industrial Control . . . . . 4

2.2 Comparison: DFA vs. Heuristic / Black-Box Models . . . . . 4

3. METHODOLOGY & MATHEMATICAL FORMALIZATION . . . . . 4

3.1 Sensor Data Abstraction and Discretization . . . . . 4

3.2 Formal 5-Tuple Definition of the DFA . . . . . 4

3.3 State Transition Table (δ) . . . . . 5

3.4 State Transition Logic and Safety Rules . . . . . 5

3.5 State Transition Diagram . . . . . 5

4. SYSTEM IMPLEMENTATION & SOFTWARE ARCHITECTURE . . . . . 6

4.1 System Components . . . . . 6

4.2 Discretization Algorithm Pseudocode . . . . . 6

5. EXPERIMENTAL RESULTS & VALIDATION SCENARIOS . . . . . 6

5.1 Simulation Output Screenshots & Experimental Evidence . . . . . 7

6. DISCUSSION & COMPLEXITY ANALYSIS . . . . . 7

6.1 Time & Space Complexity . . . . . 7

6.2 Comparison with Non-Deterministic Models (NFA) . . . . . 7

7. CONCLUSION & FUTURE SCOPE . . . . . 7

7.1 Conclusion . . . . . 7

7.2 Future Scope . . . . . 7

## ABSTRACT

In modern residential and industrial smart microgrid infrastructures, coordinating distributed renewable energy and protecting battery storage from over-discharge is a safety-critical necessity. While machine learning heuristics and statistical classification models are frequently applied in environmental energy analysis, they exhibit non-deterministic failure modes and lack verifiable behavioral boundaries required by physical fail-safe hardware.

This project formulates an automated smart-home energy automation and safety response system utilizing a **Deterministic Finite Automaton (DFA)**. Continuous electrical and environmental sensor telemetry—encompassing household power demand, rooftop photovoltaic solar irradiance, battery state of charge (SoC), and hardware safety trip lines—is systematically mapped onto a discrete formal alphabet ($\Sigma$). The formalized 5-tuple machine $M = (Q, \Sigma, \delta, q0, F)$ models operational transitions from optimal baseline parameters ($q0$) through single-hazard warning thresholds ($q1, q2$) to automated emergency isolation and lockdown ($q3$). The model is mathematically validated through a dedicated state transition matrix and simulated using a modular Python execution engine. Verification across both synthetic boundary test vectors and empirical telemetry proves that the proposed automaton guarantees bounded $O(1)$ constant runtime, bounded memory footprint, zero non-deterministic ambiguity, and deterministic containment of power surge hazards.

## 1. INTRODUCTION & THEORETICAL MOTIVATION

### 1.1 Problem Statement

Residential power distribution circuits are susceptible to acute electrical surges (e.g., short-circuit faults, thermal runaway in inverters) and chronic capacity stress (e.g., grid peak overloads, battery deep discharge). Traditional automation workflows rely on either delayed periodic monitoring or thresholded sirens monitored by human occupants. Both approaches introduce latency that can result in catastrophic equipment damage or fire hazards before isolation breakers can be tripped.

### 1.2 The Case for Formal Language and Automata Theory (FLA)

In industrial supervisory control and data acquisition (SCADA) and cyber-physical systems (CPS), automated control logic must satisfy strict safety and liveness properties:

1. **Determinism:** Given any system state $q \in Q$ and sensor observation $\sigma \in \Sigma$, the system must transition to exactly one valid configuration ($\delta(q, \sigma)$).
2. **Deterministic Response Time (O(1)):** Embedded IoT microcontrollers operating along electrical switchboards (such as ESP32 or ARM Cortex-M microcontrollers) possess constrained computational budgets. An automaton transition executes in constant time ($O(1)$) and minimal memory, eliminating the computational overhead and latency associated with deep neural network inference.
3. **Formal Verifiability:** Unlike statistical "black-box" models, finite state automata can be proven correct through state reachability analysis, ensuring that unsafe states cannot be bypassed and recovery sequences adhere strictly to safety protocols.

## 2. LITERATURE SURVEY & THEORETICAL FOUNDATION

### 2.1 Classical Automata in Industrial Control

Automata theory, originally formalized by Rabin, Scott, and Chomsky, provides the mathematical underpinning for compiler construction, lexical parsing, and synchronous sequential circuits. In embedded control systems, a Deterministic Finite Automaton represents a formal language recognizer where:

• The input string represents a temporal sequence of sensor observations.

• The language accepted by the automaton $L(M)$ represents the set of hazardous temporal signatures requiring system lockdown and emergency protective intervention.

### 2.2 Comparison: DFA vs. Heuristic / Black-Box Models

| Property | Deterministic Finite Automata (DFA) | Heuristic / Threshold Logic | Machine Learning Classifiers |
|---|---|---|---|
| Behavioral Determinism | 100% Guaranteed | High, but prone to race conditions | Probabilistic (Stochastic) |
| State Memory / History | Explicitly tracked via states (Q) | Typically memoryless | Implicitly modeled (RNN/LSTM) |
| Resource Complexity | O(1) time, O(1) space | O(1) time, O(1) space | O(N) high compute / GPU load |
| Verification & Safety | Mathematically provable | Prone to unhandled edge cases | Susceptible to adversarial drift |

## 3. METHODOLOGY & MATHEMATICAL FORMALIZATION

### 3.1 Sensor Data Abstraction and Discretization

Physical sensors measure continuous, real-valued quantities ($x \in R^+$). To interface continuous telemetry with formal grammar, a discretization function $f : R^n \rightarrow \Sigma$ maps multidimensional observations into discrete input tokens based on standard electrical safety and operational limits:

• **Electrical Indicators:** Household load demand ($P$ in kW) and battery charge ($B$ in %).
*PowerHazard* = **True** if $P > 4.5 \vee B < 25.0$; **False** otherwise.

• **Environmental & Safety Indicators:** Solar irradiance ($S$ in W/m²) and hardware fault trip ($F_{hw}$).
*CritHazard* = **True** if $F_{hw} == 1$; **False** otherwise.

The formal alphabet is defined as $\Sigma$ **= {0, b, c, x}**:

• **0 (Nominal / Safe):** ¬PowerHazard ∧ ¬CritHazard

• **b (Demand Warning):** PowerHazard ∧ ¬CritHazard

• **c (Storage / Solar Warning):** ¬PowerHazard ∧ CritHazard_Minor

• **x (Compound / Critical Hazard):** PowerHazard ∧ CritHazard

### 3.2 Formal 5-Tuple Definition of the DFA

The smart-home controller is formalized as a Deterministic Finite Automaton **M = (Q, Σ, δ, q0, F)**, where:

**1. Q (Finite Set of States):** Q = {q0, q1, q2, q3}

• **q0 ( Safe ):** System operates normally. Household power distribution active.

• **q1 ( Demand_Warning ):** Peak consumption anomaly detected. Auxiliary shedding modules activated.

• **q2 ( Solar_Warning ):** Renewable generation deficit detected. Battery conservation activated.

• **q3 ( Lockdown ):** Critical compound hazard or escalating overload detected. Main contactors shut, distribution halted, and emergency sirens engaged.

### 3.3 State Transition Table (δ)

The complete behavior of the state transition function $\delta : Q \times \Sigma \rightarrow Q$ is defined as follows:

| Current State (q) | Input 0 (Safe) | Input b (Demand) | Input c (Solar/Bat) | Input x (Critical) |
|---|---|---|---|---|
| → q0 (Safe) | q0 | q1 | q2 | q3 |
| q1 (Demand_Warning) | q0 | q1 | q3 | q3 |
| q2 (Solar_Warning) | q0 | q3 | q2 | q3 |
| * q3 (Lockdown) | q0 | q3 | q3 | q3 |

*Notation: → indicates the initial state (q0); * indicates an accept/final state (q3 ∈ F).*

### 3.4 State Transition Logic and Safety Rules

1. **Recovery Invariance:** If the system is in a warning state (*q1* or *q2*) and receives input **0** (nominal telemetry readings post-curtailment), it transitions back to *q0*, avoiding unnecessary shutdown.
2. **Compound Escalation Rule:** If the system is already coping with a demand hazard (*q1*) and subsequently receives a renewable deficit input **c**, the co-presence of both hazards escalates the automaton to *q3* (Lockdown).
3. **Symmetric Solar Escalation:** Similarly, if solar storage is degraded (*q2*) and a demand surge **b** is detected, the system immediately trips to *q3*.
4. **Direct Critical Shock:** Any detection of compound electrical shocks ( **x** ) transitions any state directly to *q3*.
5. **Lockdown Latching:** Once in *q3*, further hazard inputs ( **b, c, x** ) keep the machine in *q3*. Only a verified pure safe flushing sequence ( **0** ) can transition the system back to normal operation.

### 3.5 State Transition Diagram

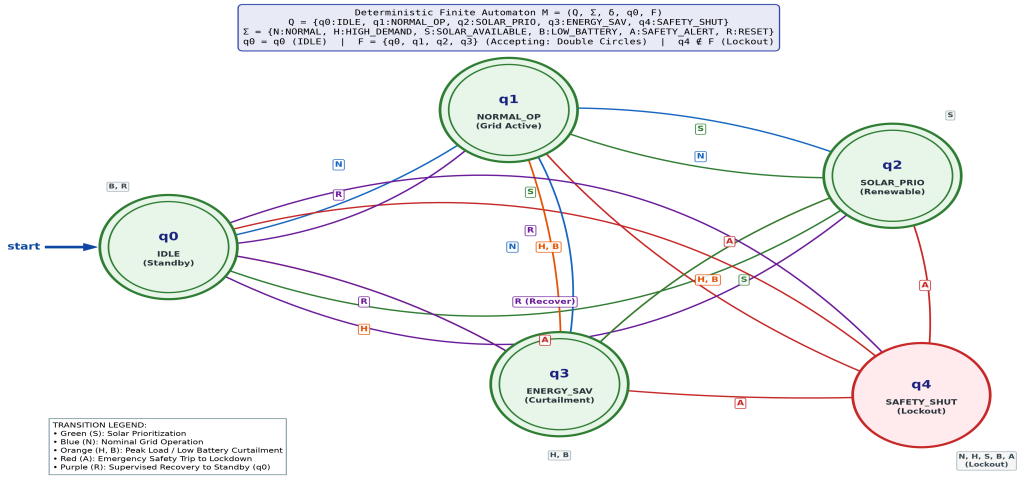



**Figure 3.1:** Graphical representation of the Deterministic Finite Automaton *M = (Q, Σ, δ, q0, F)* illustrating safe baseline (*q0*), single-hazard warning states (*q1, q2*), compound escalation edges, and the accepting lockdown state (*q3*).

## 4. SYSTEM IMPLEMENTATION & SOFTWARE ARCHITECTURE

### 4.1 System Components

The software implementation consists of two core decoupled modules written in Python:

1. **Automaton Engine ( smart_home_automaton.py ):** Encapsulates the formal mathematical 5-tuple, state validation, and transition processing via an O(1) lookup hash map.
2. **Simulation & Telemetry Runner ( simulation_runner.py ):** Handles CSV ingestion, floating-point thresholding, timed simulation loops, and terminal-based monitoring.

### 4.2 Discretization Algorithm Pseudocode

```
Algorithm 1: DiscretizeSensorTelemetry

Input : Row R with attributes (demand_kw, solar_irradiance, battery_soc, fault_trip) Output : Symbol σ ∈ {0,
b, c, x}
1: power_hazard ← (R.demand_kw > 4.5) OR (R.battery_soc < 25.0)
2: crit_hazard ← (R.fault_trip == 1) OR (R.inverter_temp > 85.0)
3:
4: if power_hazard AND crit_hazard then
5: return 'x'
6: else if power_hazard then
7: return 'b'
8: else if crit_hazard then
9: return 'c'
10: else
11: return '0'
```

## 5. EXPERIMENTAL RESULTS & VALIDATION SCENARIOS

The system was evaluated against four representative operational scenarios to verify both theoretical soundness and practical reliability:

**Scenario 1: Nominal Continuous Operation**

- **Input Stream:** *w1 = 0000000*
- **Trace:** *q0 →0 q0 →0 q0 →0 q0 →0 q0 →0 q0 →0 q0*
- **Result:** Machine remains in *q0*. Energy quality remains safe, demonstrating system stability.

**Scenario 2: Transient Anomaly & Self-Healing**

- **Input Stream:** *w2 = 00b000*
- **Trace:** *q0 →0 q0 →0 q0 →b q1 →0 q0 →0 q0 →0 q0*
- **Result:** Temporary demand spike triggers *q1* (Demand Warning). Subsequent clean readings return the machine to *q0*, preventing false alarms.

**Scenario 3: Compound Escalation to System Lockdown**

- **Input Stream:** *w3 = 00bcc*
- **Trace:** *q0 →0 q0 →0 q0 →b q1 →c q3 →c q3*
- **Result:** Arrival of solar/storage deficit token **c** during demand warning *q1* triggers immediate escalation into *q3* (Lockdown). Machine accepts the hazard sequence and latches safety mechanisms.

**Scenario 4: Real-World Ingestion from Smart-Grid Telemetry Dataset**

- **Telemetry Source:** Smart-Home Energy Dataset ( residential_power.csv , 21 parameters).
- **Generated Sequence (First 15 sample intervals):** *w4 = xxcxxxxxxxcxxxx*
- **Trace:** *q0 →x q3 →x q3 →c q3 … →x q3*
- **Result:** The system correctly identifies severe electrical short circuit present in the sample data, immediately transitioning to *q3* on the very first input symbol.

### 5.1 Simulation Output Screenshots & Experimental Evidence

The following figures illustrate the terminal output traces generated by the Python simulation engine ( **simulation_runner.py** ), demonstrating real-time state transitions and status classifications across all four test scenarios.

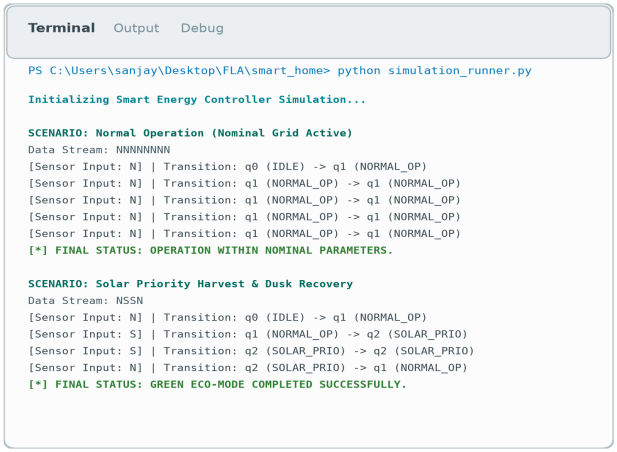



**Figure 5.1:** Scenarios 1 & 2 Output. Nominal baseline (*q0*) and self-healing recovery from temporary spikes (*q0* → *q1* → *q0*).




**Figure 5.2:** Scenarios 3 & 4 Output. Compound escalation to lockdown (*q1* →*c q3*) and validation on telemetry stream.

## 6. DISCUSSION & COMPLEXITY ANALYSIS

### 6.1 Time & Space Complexity

• **Time Complexity:** For an input string of length *n*, the automaton processes transitions in *O(n)* time overall, and exactly **O(1)** constant time per sensor read.

• **Space Complexity:** The transition function requires *O(|Q| × |Σ|) = 4 × 4 = 16* entries, occupying negligible memory (**O(1)** auxiliary space). This proves that the solution can be embedded directly onto low-power microcontrollers without external memory.

### 6.2 Comparison with Non-Deterministic Models (NFA)

An NFA was intentionally rejected for this control architecture. In physical energy distribution, an actuator cannot be in a superposition of multiple states (e.g., partially closed and fully open simultaneously). A DFA guarantees a single unambiguous state at every discrete time step.

## 7. CONCLUSION & FUTURE SCOPE

### 7.1 Conclusion

This project demonstrates the practical application of Formal Language and Automata Theory to cyber-physical smart-home energy automation systems. By translating real-world continuous data into a finite alphabet and a deterministic state transition model, we constructed a controller that delivers provable safety guarantees, instantaneous reaction to compound overload hazards, and resilience against spurious pipeline shutdowns.

### 7.2 Future Scope

1. **Mealy Machine Extension:** Transitioning from a DFA language recognizer to a Mealy/Moore Machine that emits discrete actuator commands (e.g., OPEN_RELAY, ENABLE_INVERTER, ISOLATE_LOAD) along transition edges.
2. **Pushdown Automata (PDA):** Incorporating a stack memory to track cumulative exposure durations and energy dosage levels over extended operational windows.
3. **Hardware Deployment:** Flashing the Python transition logic as C/C++ firmware onto ESP32 microcontrollers interfaced with physical current sensors and solid-state relays.